

ITE 1942 – ICT PROJECT

PROJECT REPORT

Level 01

Rent Master - Car Rental Management System

Submitted by:

PATHIRANA SMV

E2248422

Bachelor of Information Technology (External Degree)

Faculty of Information Technology

University of Moratuwa

Table of Contents

1. INTRODUCTION.....	1
2. RELATED WORK.....	3
3. SYSTEMS ANALYSIS.....	7
4. SYSTEM DESIGN.....	11
5. SYSTEM IMPLEMENTATION.....	20
6. APPENDIX	39
7. REFERENCES.....	51

1. Introduction

1.1 Background & Motivation

In today's swiftly changing world, efficient and reliable transportation is crucial for personal and business activities. Car rental services play a significant role in providing mobility solutions for individuals who do not own vehicles, travelers in need of temporary transportation, and businesses requiring fleet vehicles. Despite the high demand, many car rental businesses struggle with managing their operations effectively due to the lack of a robust system. The primary need is for an automated system that simplifies and streamlines the car rental process. Manual methods or outdated software often led to issues such as double bookings, inaccurate record keeping, and inefficient customer service. These problems not only cause frustration for customers but also lead to financial losses and operational inefficiencies for the rental companies.

For customers, a disorganized car rental process can result in booking errors, and an overall unsatisfactory experience. This can lead to frustration, inconvenience, and a negative perception of the rental company. On the other hand, for rental companies, these issues can have far reaching consequences. They may experience decreased customer loyalty as dissatisfied customers turn to competitors, resulting in a loss of revenue. Additionally, dealing with manual errors and inefficient processes can lead to increased operational costs and decreased profitability. Therefore, there is a clear need for a solution that enhances the efficiency and accuracy of car rental operations to improve the experience for both customers and rental companies alike.

Several car rental management systems are available, ranging from manual paper works, basic software to comprehensive platforms. However, many of these solutions are either too expensive for small businesses or lack the specific features required by different rental agencies. A tailored solution that addresses the needs of car rental agencies is often missing.

1.2 Problem in Brief

The problem addressed in this project is the inefficiency and complexity involved in managing car rental operations. This includes managing car registrations, customer registrations, car rental and returning processes. Current manual processes or inadequate software led to issues such as booking conflicts, inefficient handling of customer data, and difficulties in tracking vehicle availability.

To solve this problem, this project aims to develop a comprehensive car rental management system. This desktop application will automate and integrate all aspects of the car rental process, providing a user-friendly interface for managing car and customer registrations, rentals, and returns. The system will ensure accurate record-keeping, real-time availability updates, and improved customer service.

1.3 Aims & Objectives

The aim of this project is to develop a system for addressing the inefficiencies and complexities in car rental management by creating a desktop application.

The objectives of the system are as follows.

- Do a critical analysis of system requirements by conducting a thorough examination of current car rental processes to identify specific requirements and challenges.
- Design and develop a system for solving the problem by creating a detailed design and a user-friendly desktop application that automates car and customer registrations, as well as car rentals and returns.
- Evaluate the proposed system by thoroughly testing it to ensure it meets all requirements and performs reliably under various conditions and make necessary adjustments to improve the system.

1.4 Summary

Many car rental businesses struggle due to outdated systems, leading to issues like booking errors and decreased customer satisfaction. This project aims to develop a user-friendly desktop application to streamline car rental operations, addressing challenges such as manual processes and inadequate software. By automating tasks like car and customer management and improving record keeping, the system seeks to enhance efficiency and customer service, ultimately benefiting both rental companies and customers.

2. Related Work

The car rental industry has seen significant growth over the past years, necessitating the development of efficient management systems to handle the increasing demand. This chapter explores the challenges associated with car rental management and reviews existing solutions. By examining these solutions, we can identify gaps and areas for improvement that will inform the development of a new, more effective car rental management system.

2.1 Challenges in Car Rental Management

Car rental management encompasses various complex processes, each presenting unique challenges. Some of the key challenges and solutions associated with the system are mentioned below.

- Booking conflicts are a common occurrence, primarily stemming from double bookings or overbooking, which frequently arise due to manual errors or a lack of real-time updates in the system. Ensuring that a vehicle is available as promised to the customer is paramount to maintaining trust and satisfaction. Therefore, resolving these conflicts efficiently is vital to enhancing the overall customer experience and fostering positive relationships with clients.

- Efficient management of customer data is indispensable for car rental businesses to provide personalized services and maintain customer satisfaction. Accurate records of customer information facilitate smooth transactions and enable rental companies to cater to individual preferences effectively. However, manual data entry processes are prone to errors, which can lead to billing inaccuracies, scheduling conflicts, and customer dissatisfaction. Adopting automated systems for customer data management enhances accuracy, streamlines operations, and improves overall service quality.
- Effective management of vehicle data is crucial for ensuring the availability, maintenance, and safety of rental fleets. Accurate records of vehicle information, including maintenance history, usage logs, and availability status, enable rental companies to optimize fleet utilization and meet customer demands efficiently. Manual data entry increases the risk of errors, such as incorrect maintenance schedules or double bookings, which can disrupt operations and impact customer experience negatively. Implementing automated systems for vehicle data management enhances accuracy, minimizes scheduling conflicts, and ensures the seamless operation of car rental services.
- Pricing and billing present significant challenges for car rental companies, particularly when implementing dynamic pricing models based on various factors such as demand, vehicle type, rental duration, and additional services. Managing these dynamic pricing structures can be complex and requires careful consideration to ensure competitiveness while maximizing revenue. Furthermore, accurate billing and transparent invoicing are essential to maintain customer satisfaction and trust. Any discrepancies or errors in the billing process can lead to customer dissatisfaction and tarnish the reputation of the rental company. Therefore, implementing robust pricing and billing systems that accurately reflect rental charges and provide transparent invoicing is crucial to ensure a positive customer experience and uphold the integrity of the rental service.

- Providing a seamless and user-friendly experience for users is critical for operational efficiency. A well designed and intuitive interface simplifies the rental process, reducing the likelihood of errors and speeding up operations. By incorporating feedback and continuously refining the user experience, rental companies can optimize customer interactions and foster loyalty. Additionally, a positive user experience enhances brand reputation and encourages repeat business, further contributing to the company's success.

2.2 Existing Solutions

Several solutions have been developed to address the challenges in car rental management. This section reviews some of the systems and their approaches to handling these challenges.

- **Manual Paperwork Systems**
Manual paperwork systems have traditionally been used by car rental companies to manage their operations. These systems involve extensive record-keeping through physical documents and ledgers. While this method can be straightforward for very small operations, it often leads to significant inefficiencies and errors. Manual entry is time-consuming and prone to mistakes, such as double bookings, inaccurate billing, and overlooked maintenance schedules. Additionally, the lack of real-time updates means that information can quickly become outdated, leading to customer dissatisfaction and operational disruptions. As businesses grow, the limitations of manual paperwork become increasingly apparent, highlighting the need for more efficient and accurate solutions.

- **Overpriced Software Solutions**

On the other end of the spectrum, there are comprehensive software solutions that offer a wide range of features for car rental management. While these systems can be very effective in automating processes and providing real-time updates, they often come with a high price tag. Small and medium-sized rental companies may find these solutions prohibitively expensive, making it difficult for them to justify the investment. Additionally, these software packages may include a range of features that are not necessarily needed by all businesses, leading to complexity and underutilization. This mismatch between cost, complexity, and actual business needs often results in companies either struggling with manual methods or overspending on solutions that do not fully meet their requirements.

This built system will efficiently handle these tasks, offering a balanced solution between manual paperwork and overpriced software. By automating vehicle and customer data management , rental bookings, and returning processes, the system eliminates the errors and inefficiencies associated with manual methods. It provides real-time updates to prevent double bookings and ensures accurate billing . Additionally, the system is designed to be user-friendly and tailored to the specific needs of small and medium sized rental companies, avoiding the complexity and unnecessary features of overpriced solutions. This approach ensures that businesses can manage their operations smoothly while enhancing customer satisfaction and loyalty.

3.System Analysis

The development of an effective car rental management system requires a thorough understanding of both the functional and non-functional requirements. These requirements will ensure that the system meets the needs of its users and operates efficiently under various conditions. This section presents all the requirements addressed in this project in detail.

3.1.1 System Requirements

- User Management and Security

This built system facilitates comprehensive user management by management of admin users enhancing security and operational efficiency. Security of data is maintained, with secure user authentication processes to log into the system.

- Vehicle Management

Vehicle management is streamlined through efficient vehicle registration and categorization processes. The system tracks vehicle availability in real-time, ensuring that the rental fleet is optimally utilized. This reduces the risk of double bookings and ensures that customers always have accurate information about vehicle availability, enhancing their rental experience.

- Customer Management

The system streamlines customer management through efficient customer registration and profile management. It enables quick and accurate data entry, maintaining detailed customer profiles that include personal information. This capability allows for personalized customer service, improved customer retention, and more effective marketing efforts.

- Booking Management

Booking management is enhanced with real-time updates on bookings and availability, significantly reducing errors and customer frustration. The system efficiently handles booking conflicts, ensuring a smooth and reliable process for both the rental company and its customers. This leads to improved customer satisfaction and streamlined operations.

- **Rental Process**

The rental process is made more efficient with streamlined check-in and check-out procedures. Accurate pricing ensured, reducing the likelihood of errors and disputes. The system also effectively manages late returns and associated fees, ensuring that all financial aspects of the rental process are handled transparently and accurately.

3.2 Functional Requirements

- **Secure and Efficient Login Process**

By ensuring a secure log in process, the system maintains the integrity of user data and upholds the confidentiality of sensitive information.

- **Vehicle Registration and Categorization**

The system facilitates the registration of new vehicles, capturing detailed information such as make, model, year, and category. It also enables the categorization of vehicles by the model. This organized approach to vehicle registration and categorization helps in managing them more effectively, ensuring that customers can easily find and rent the vehicle that meets their needs.

- **Customer Registration and Profile Management**

The system streamlines the process of customer registration by capturing essential personal details. Additionally, it allows for the ongoing management and updating of customer profiles, ensuring that the information remains current and accurate. This feature supports personalized customer service and efficient handling of customer data, enhancing overall customer satisfaction.

- **Vehicle Availability Tracking**

Real-time tracking of vehicle availability is a key feature of the system. It provides real-time information on the status of each vehicle, whether it is rented, returned, or

available for booking. Availability status ensure that the rental company can manage its fleet dynamically, reducing the chances of double bookings and improving operational efficiency.

- **Vehicle Renting and Return Processes**

The system offers streamlined procedures for both vehicle check-in and check-out processes, optimizing the rental experience for users. For vehicle pickup, detailed and efficient process is provided. Similarly, a detailed functionality is implemented for vehicle return.

- **Accurate Billing and Pricing**

The system ensures precise billing and invoicing by calculating rental charges based on factors such as rental duration, vehicle type, and additional services. This feature minimizes the risk of billing discrepancies and disputes, enhancing transparency and customer satisfaction. Additionally, it handles extra charges for late returns and other additional fees, providing a comprehensive and reliable billing process.

- **Earnings Overview**

The system provides a comprehensive overview of earnings, allowing users to easily track and analyze revenue generated over specific periods. Additionally, an overview of all earnings is available in a tabular format for quick reference and analysis.

3.3 Non-functional Requirements

- **Scalability**

This system has the capability to handle increasing numbers of users and vehicles without experiencing performance degradation. Efficient database management is implemented to support large volumes of data and ensure optimal system performance under varying loads.

- **Response Time**

User actions are processed within acceptable time limits to provide a responsive and seamless user experience. Real-time updates, such as changes in vehicle availability, are reflected immediately in the system to ensure accuracy and timeliness.

- **Usability**

The system features an intuitive and user-friendly interface, facilitating ease of use for all users. Consistent design and layout across the interface help to reduce the learning curve for new users, ensuring efficient navigation and interaction with the system.

- **Reliability**

This system demonstrates reliability in its operation, maintaining consistent performance and uptime. This reliability ensures that users can depend on the system to be available and functional whenever needed.

- **Security**

The system prioritizes data protection measures to safeguard sensitive information from unauthorized access. Secure login process is implemented to prevent data breaches and maintain data integrity.

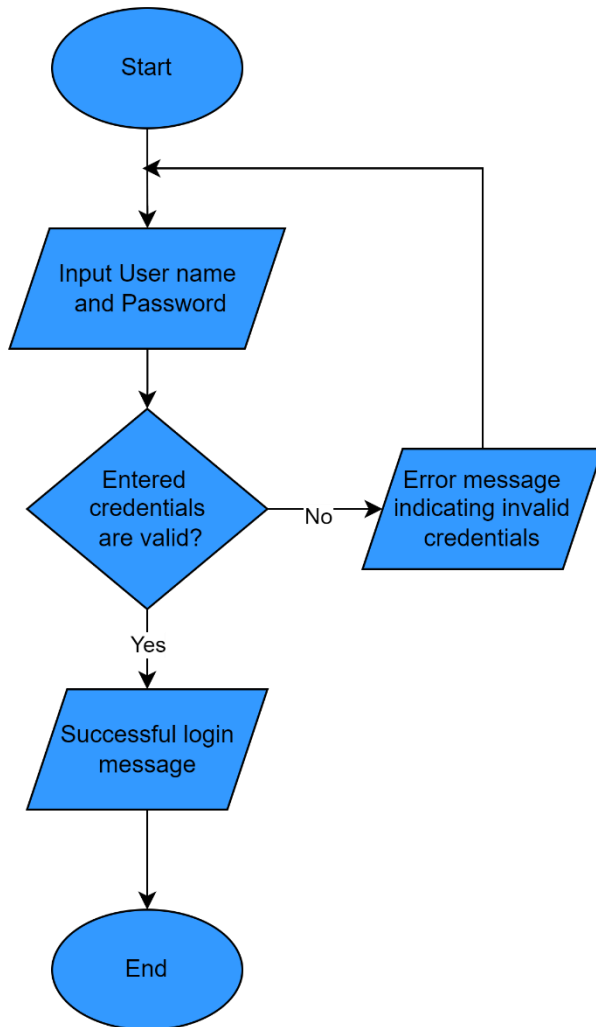
3.4 Summary

This chapter provided a comprehensive analysis of the functional and non-functional requirements for the car rental management system. The functional requirements detailed the specific actions and processes the system must perform, while the non-functional requirements addressed the quality attributes necessary for the system's performance, usability, reliability, security, and maintainability.

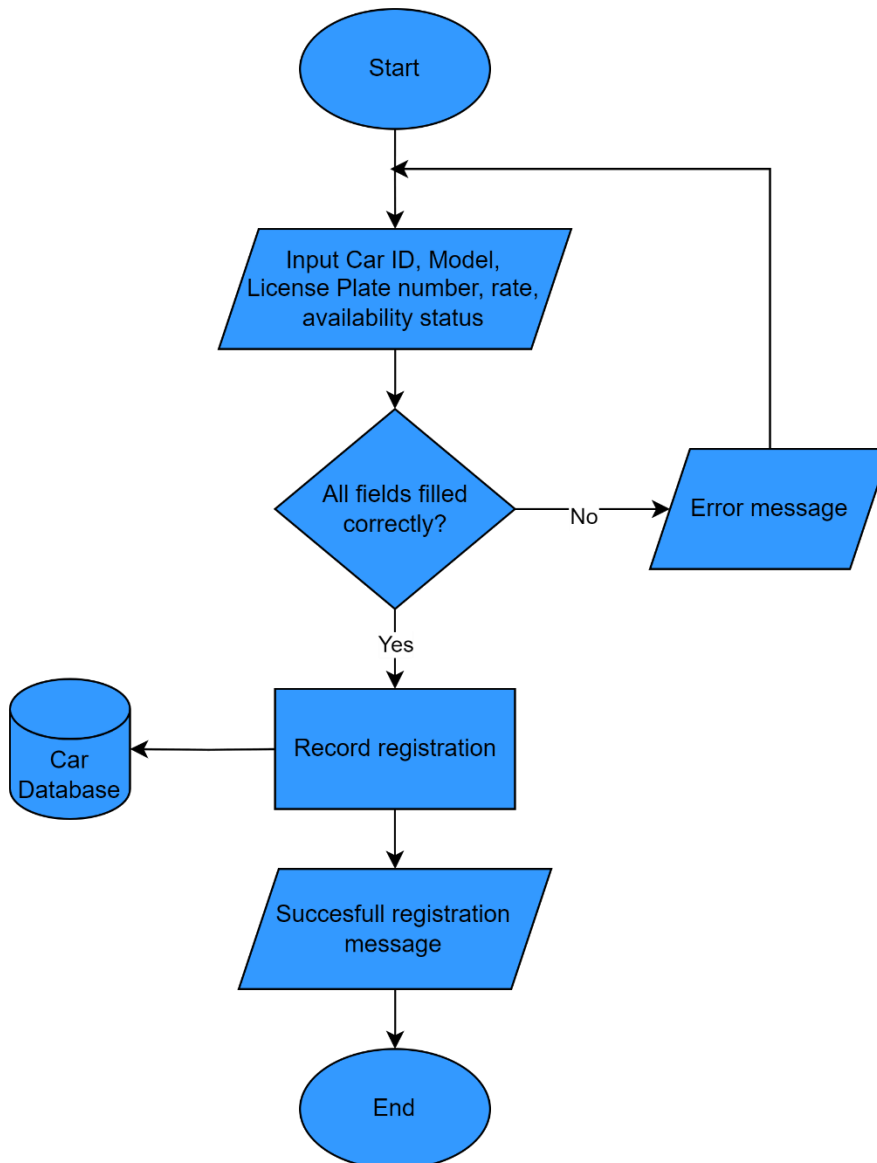
4. System Design

4.1 Flow Charts

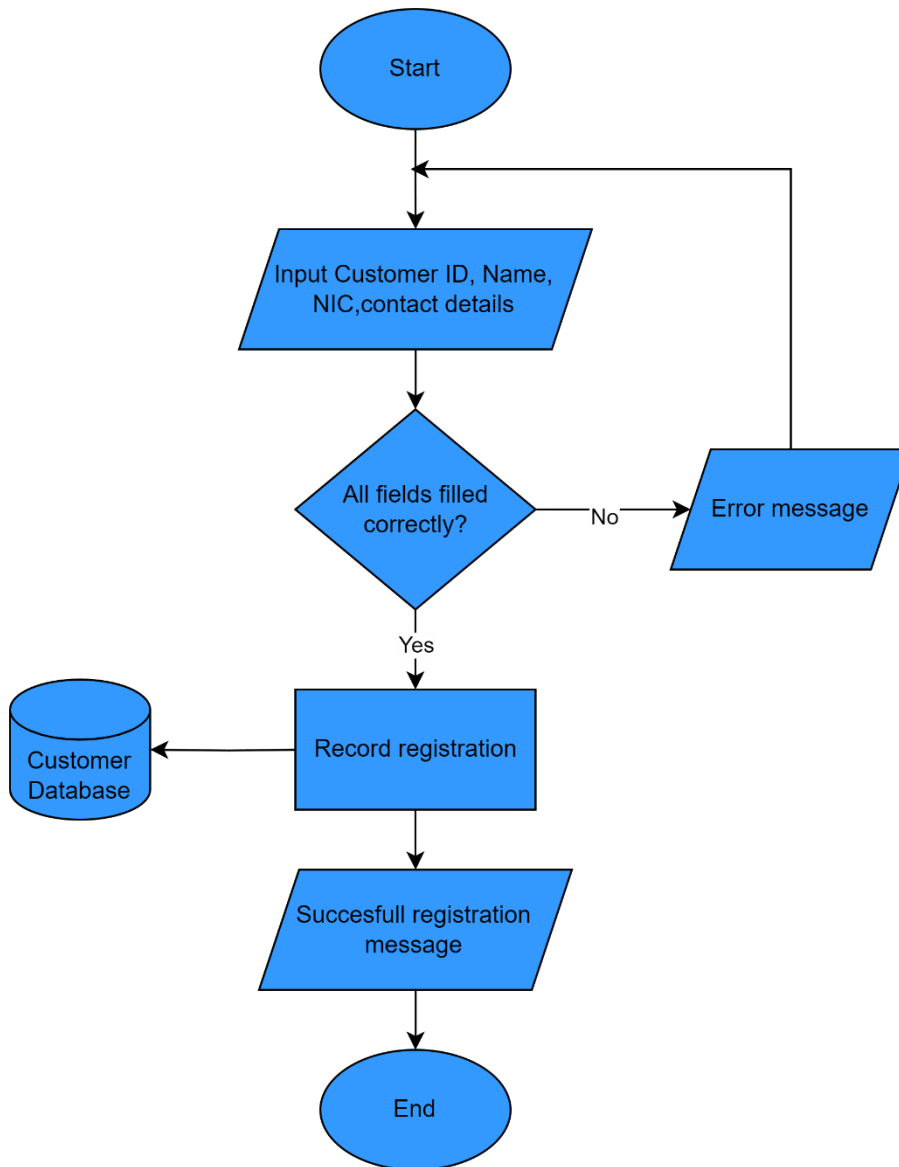
- Flow chart for the user login process



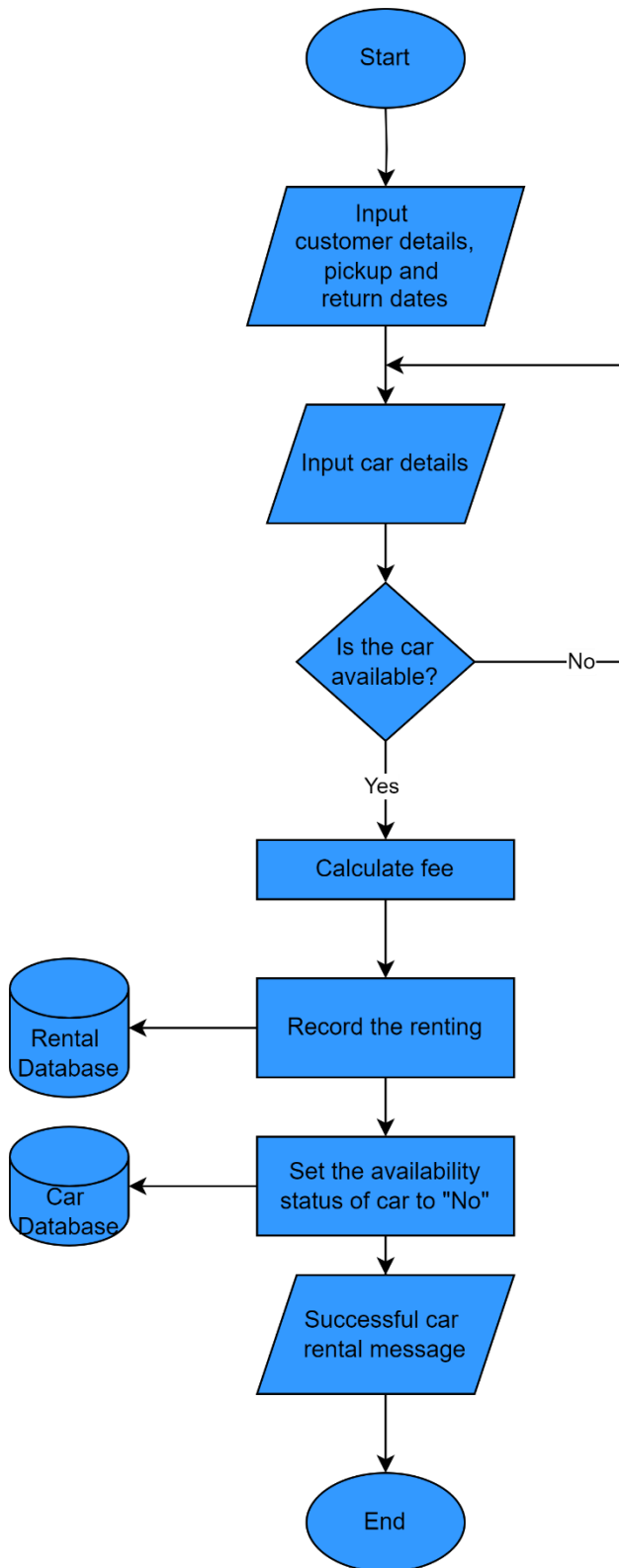
- Flow chart for the car registration process.



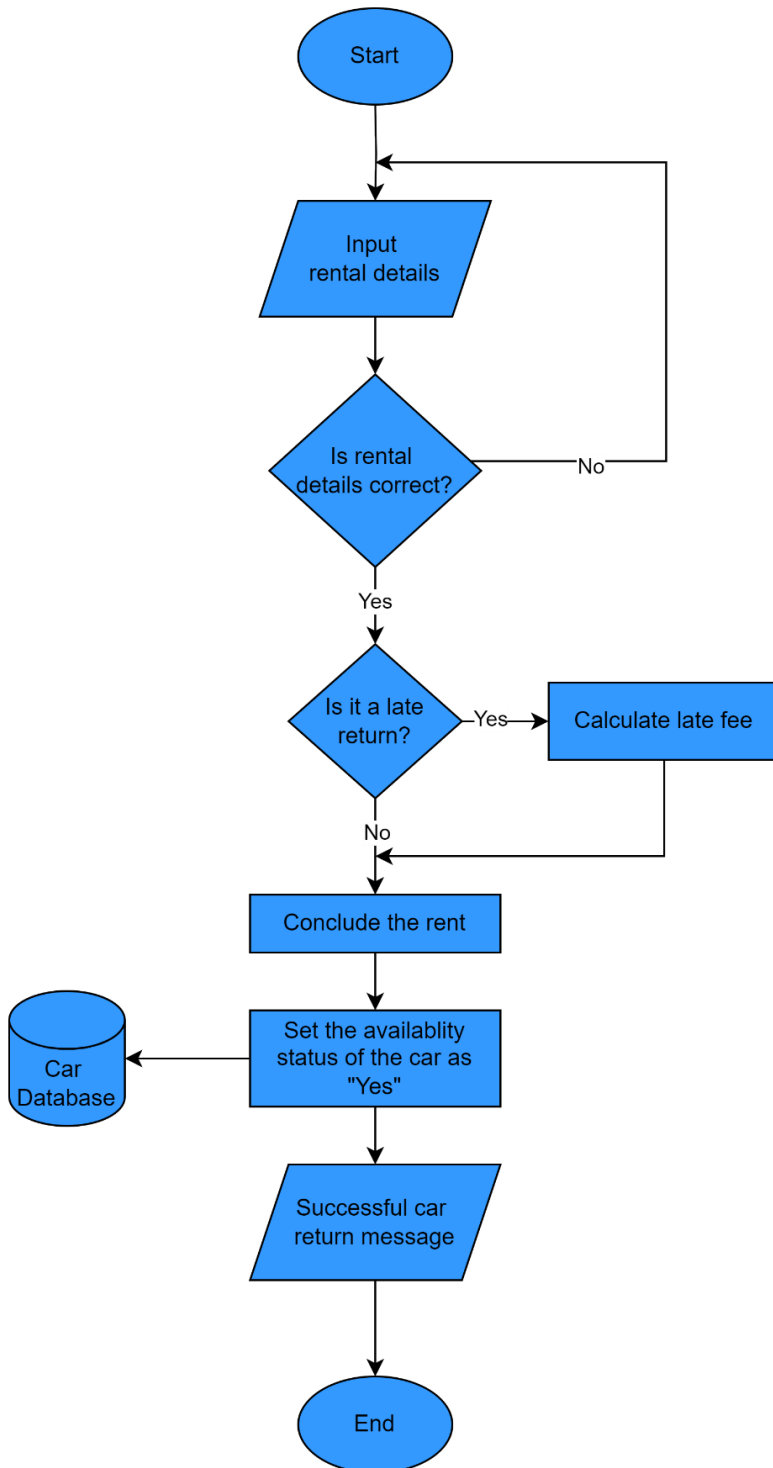
- Flow chart for the customer registration process.



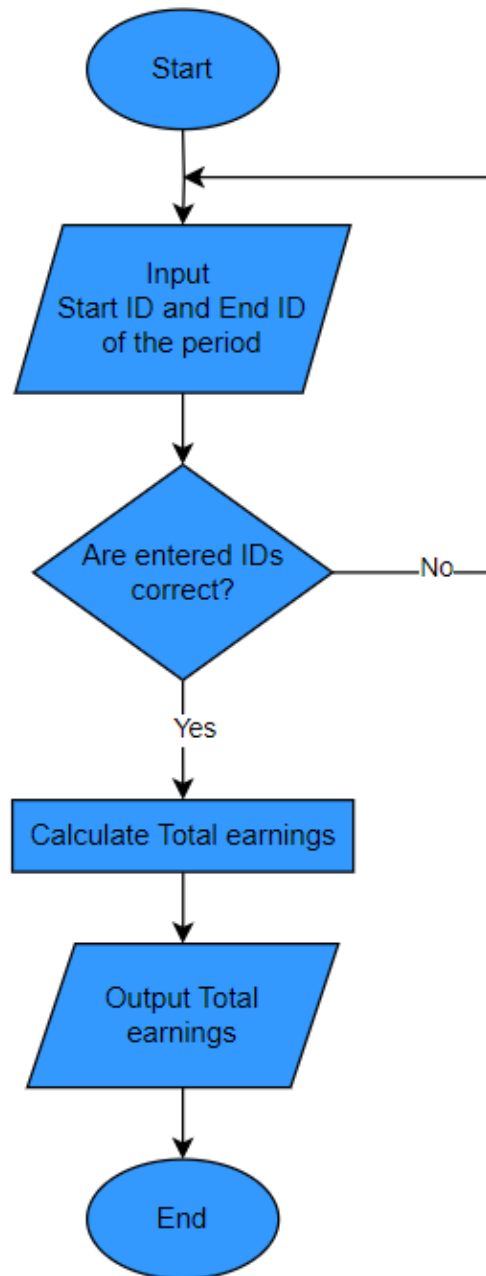
- Flow chart for the rental commencement.



- Flow chart for the rental concluding process.



- Flow chart for calculating total earnings for a given period.



4.2 Pseudo Codes

- Pseudocode for user login

```
INPUT username and password
IF username and password not valid THEN
    DISPLAY "Invalid request"
ELSE
    DISPLAY "Successful login"
END IF
END
```

- Pseudocode for car registration

```
INPUT car id, model ,license plate number , availability status
IF all fields are filled
    Add car to database
    DISPLAY "Customer registered successfully!"
ENDIF
ELSE
    DISPLAY "Fill out all the fields"
END
```

- Pseudocode for customer registration

```
INPUT customer id, name ,NIC , contact
IF all fields are filled
    Add customer to database
    DISPLAY "Customer registered successfully!"
ENDIF
```

```
ELSE
    DISPLAY "Fill out all the fields"
END
```

- Pseudocode for rental commencement

```
INPUT customer details, pickup and return date
INPUT car details
IF car is not available
    DISPLAY "Car is not available!"
    RETURN
END IF
ELSE
    Calculate rental fee
    Create new rental record
    SET car availability to "No"
    DISPLAY "Car rented successfully! "
END
```

- Pseudocode for rental concluding process

```
INPUT rental details
IF rental not found
    DISPLAY "No ongoing rental found for this customer and car!"
    RETURN
END IF
ELSE
    IF late return
```

```
        Calculate late fee
    END IF
    SET car availability to "Yes"
    DISPLAY "Rental concluded successfully! "
END
```

- Pseudocode for calculate earnings of a given period

```
INPUT start id, end id
IF valid range
    Calculate total earnings
    DISPLAY total earnings
END IF
ELSE
    DISPLAY "Select a valid range"
END
```

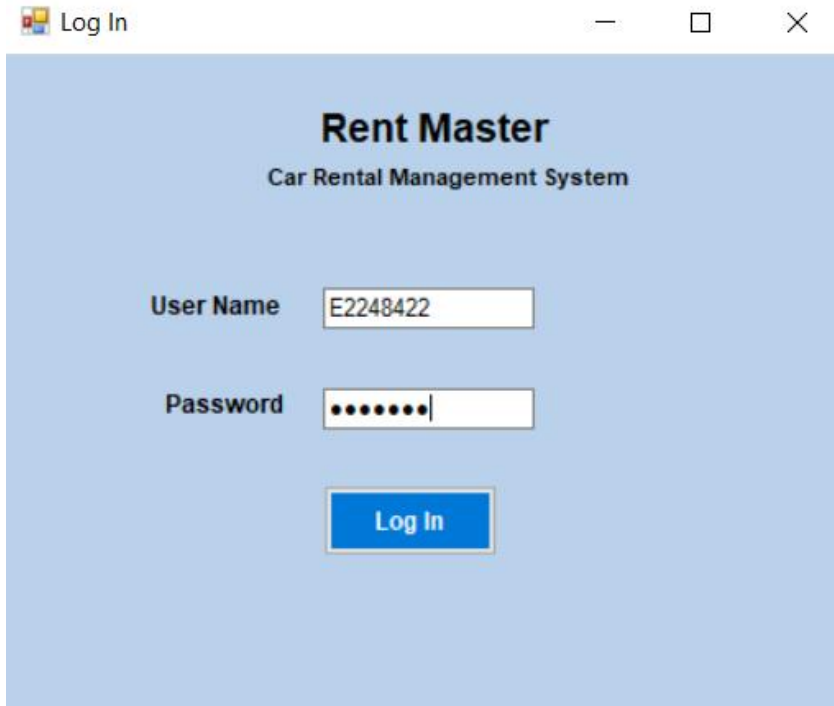
4.3 Summary

This chapter provided a comprehensive design of the solution, showcasing how each functional requirement is modeled using flowcharts and pseudocodes. The chapter is structured to include detailed and complex diagrams, ensuring that the design is thoroughly and effectively communicated. By integrating detailed flowcharts and pseudocodes, this chapter effectively captured the design of the solution, ensuring all functional requirements were methodically addressed.

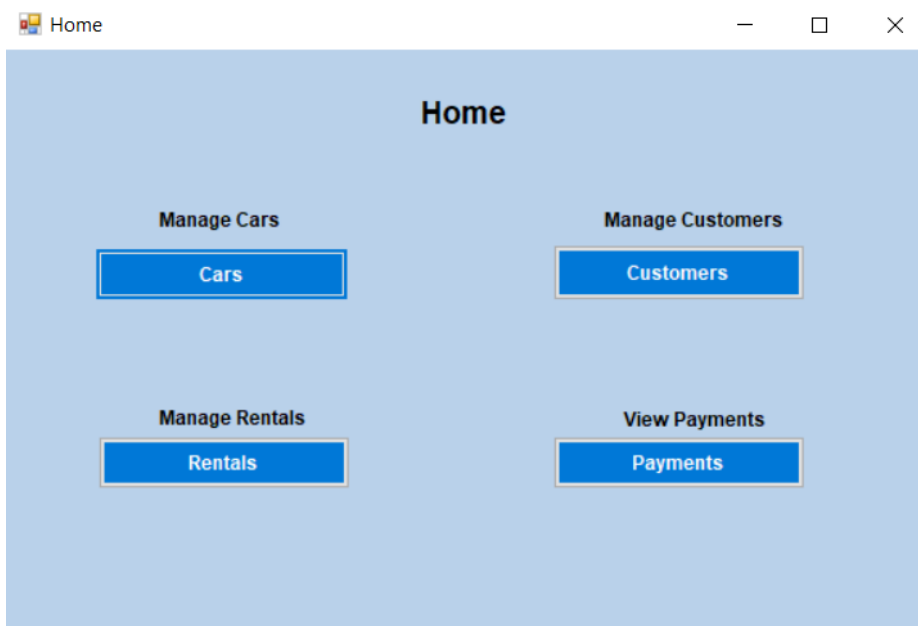
5. System Implementation

5.1 User Login Process

The user must enter the correct username and password to login to the system.



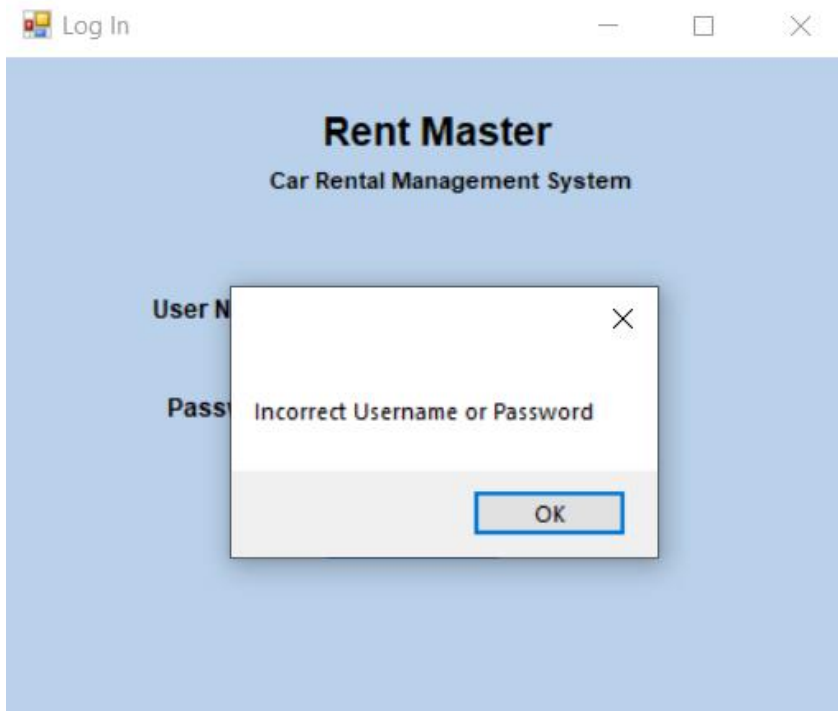
The screenshot shows a window titled "Log In" with a light blue background. At the top center, the text "Rent Master" is displayed in a large, bold, black font, with "Car Rental Management System" in a smaller font below it. There are two input fields: "User Name" containing the text "E2248422" and "Password" containing seven dots. Below these fields is a blue button with the text "Log In" in white. The window has standard Windows window controls (minimize, maximize, close) in the top right corner.



The screenshot shows a window titled "Home" with a light blue background. At the top center, the text "Home" is displayed in a large, bold, black font. Below this, there are four blue buttons arranged in a 2x2 grid. The top-left button is labeled "Cars" and is under the heading "Manage Cars". The top-right button is labeled "Customers" and is under the heading "Manage Customers". The bottom-left button is labeled "Rentals" and is under the heading "Manage Rentals". The bottom-right button is labeled "Payments" and is under the heading "View Payments". The window has standard Windows window controls (minimize, maximize, close) in the top right corner.

After a successful login, the user will be directed to the application's home window, as shown above.

If the login credentials are incorrect, a popup window will appear, prompting the user to enter the correct credentials.



Below is the code for the login process.

```
SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=D:\\Projects\\CarRental\\CarRent
al\\CarRentalDB.mdf; Integrated Security=True;");
SqlCommand cmd;
SqlDataReader dr;

private void logBtn_Click(object sender, EventArgs e)
{
    string userName = userTxt.Text;
    string password = pwTxt.Text;

    cmd = new SqlCommand("SELECT * FROM login WHERE username = @username AND
password = @password", con);
    cmd.Parameters.AddWithValue("@username", userName);
    cmd.Parameters.AddWithValue("@password", password);

    con.Open();
    SqlDataReader dr = cmd.ExecuteReader();

    if (dr.Read())
    {
```

```

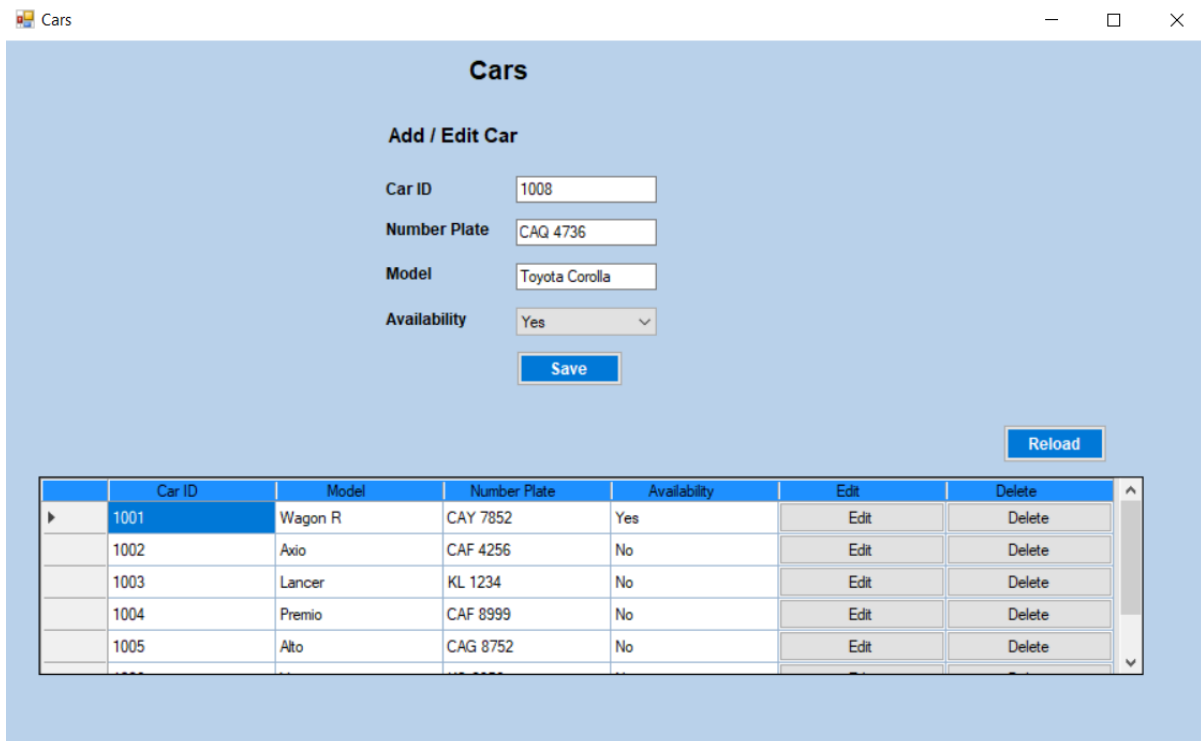
        dr.Close();
        con.Close();
        this.Hide();
        Home home = new Home();
        home.Show();
    }
    else
    {
        dr.Close();
        con.Close();
        MessageBox.Show("Incorrect Username or Password");
    }
}

```

5.2 Car Management

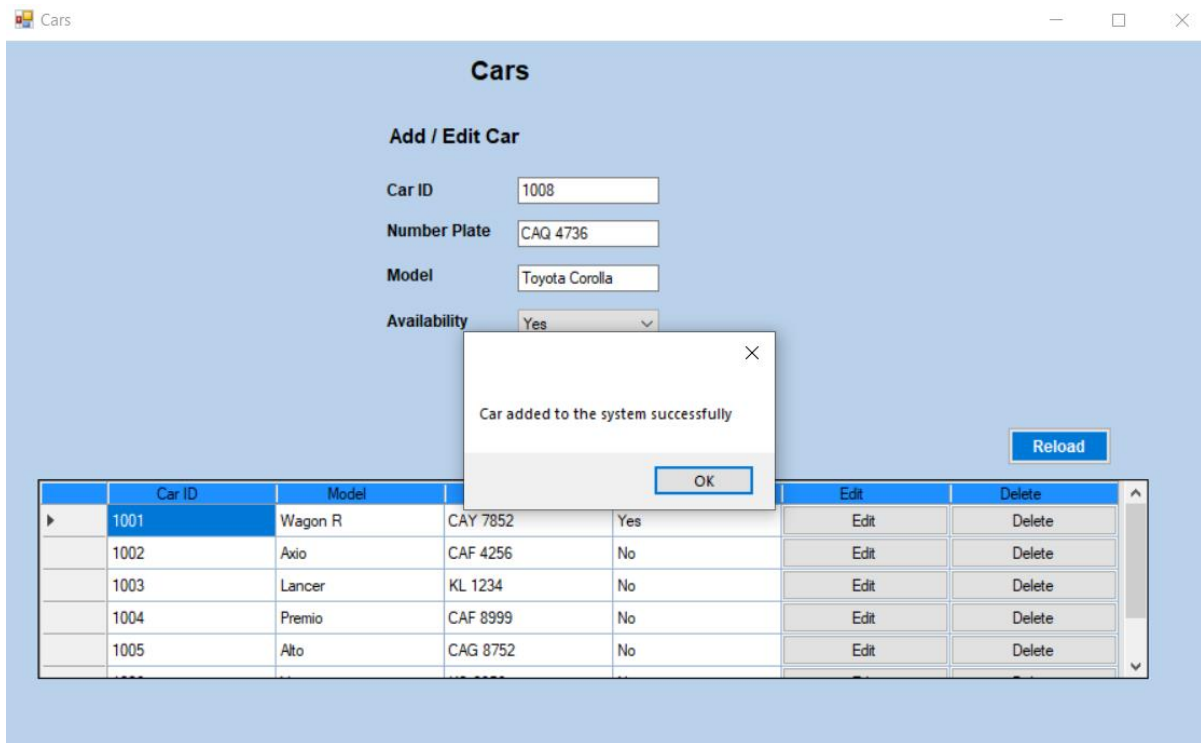
5.2.1 Add a New Car to the System (Car Registration)

A new car can be added to the system by inputting the car model, license plate number, availability status and clicking on the Save button. The Car ID is automatically generated.



The screenshot shows a Windows application window titled "Cars". Inside, there's a section titled "Add / Edit Car" with four input fields: "Car ID" (containing "1008"), "Number Plate" (containing "CAQ 4736"), "Model" (containing "Toyota Corolla"), and "Availability" (a dropdown menu set to "Yes"). Below these fields is a blue "Save" button. To the right of the form is a blue "Reload" button. At the bottom, there's a table with 7 columns: Car ID, Model, Number Plate, Availability, Edit, and Delete. The table contains 5 rows of data.

Car ID	Model	Number Plate	Availability	Edit	Delete
1001	Wagon R	CAY 7852	Yes	Edit	Delete
1002	Axio	CAF 4256	No	Edit	Delete
1003	Lancer	KL 1234	No	Edit	Delete
1004	Premio	CAF 8999	No	Edit	Delete
1005	Alto	CAG 8752	No	Edit	Delete



After a successful operation, a popup window will be shown to notify the user, and the new car will now be visible on the table.

	Car ID	Model	Number Plate	Availability	Edit	Delete
	1004	Premio	CAF 8999	No	Edit	Delete
	1005	Alto	CAG 8752	No	Edit	Delete
	1006	Vitz	KO 2356	No	Edit	Delete
	1007	Honda Civic	CAH 9874	Yes	Edit	Delete
▶	1008	Toyota Corolla	CAQ 4736	Yes	Edit	Delete

5.2.2 Edit Car Details

Details of an existing car in the system can be updated by clicking on the edit button. The details of the selected car will be displayed, and after updating them, the user has to click the save button.

Cars

Cars

Add / Edit Car

Car ID

1006

Number Plate

KO 2356

Model

Vitz

Availability

Yes

Save

Reload

	Car ID	Model	Number Plate	Availability	Edit	Delete
	1004	Premio	CAF 8999	No	Edit	Delete
	1005	Alto	CAG 8752	No	Edit	Delete
▶	1006	Vitz	KO 2356	No	Edit	Delete
	1007	Honda Civic	CAH 9874	Yes	Edit	Delete
	1008	Toyota Corolla	CAQ 4736	Yes	Edit	Delete

Cars

Cars

Add / Edit Car

Car ID

1006

Number Plate

KO 2356

Model

Vitz

Availability

Yes

Car details updated successfully

OK

Reload

	Car ID	Model	Number Plate	Availability	Edit	Delete
	1004	Premio	CAF 8999	No	Edit	Delete
	1005	Alto	CAG 8752	No	Edit	Delete
▶	1006	Vitz	KO 2356	No	Edit	Delete
	1007	Honda Civic	CAH 9874	Yes	Edit	Delete
	1008	Toyota Corolla	CAQ 4736	Yes	Edit	Delete

After a successful operation, a popup window will be shown to notify the user, and the updated details will be visible on the table.

	Car ID	Model	Number Plate	Availability	Edit	Delete
	1004	Premio	CAF 8999	No	Edit	Delete
	1005	Alto	CAG 8752	No	Edit	Delete
▶	1006	Vitz	KO 2356	Yes	Edit	Delete
	1007	Honda Civic	CAH 9874	Yes	Edit	Delete
	1008	Toyota Corolla	CAQ 4736	Yes	Edit	Delete

The following code is used for adding new cars to the system and editing the details of existing cars in the system.

```
private void carAddBtn_Click(object sender, EventArgs e)
{
    string regNo = carIDTxt.Text;
    string numPlate = npTxt.Text;
    string model = modelTxt.Text;
    string available = avaTxt.SelectedItem.ToString();

    con.Open();
    cmd = new SqlCommand("SELECT COUNT(*) FROM cars WHERE reg_no = @reg_no",
con);
    cmd.Parameters.AddWithValue("@reg_no", regNo);
    int count = (int)cmd.ExecuteScalar();
    con.Close();

    if (insert == true)
    {
        if (count > 0)
        {
            MessageBox.Show("Car already exists with this registration
number.");
            return;
        }
        con.Open();
        cmd = new SqlCommand("INSERT INTO cars (reg_no, num_plate, model,
available) VALUES (@reg_no, @num_plate, @model, @available);", con);
        cmd.Parameters.AddWithValue("@reg_no", regNo);
        cmd.Parameters.AddWithValue("@num_plate", numPlate);
        cmd.Parameters.AddWithValue("@model", model);
        cmd.Parameters.AddWithValue("@available", available);
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Car added to the system successfully");
    }
    else
    {
        con.Open();
        cmd = new SqlCommand("UPDATE cars SET num_plate = @num_plate, model =
@model, available = @available WHERE reg_no = @reg_no;", con);
        cmd.Parameters.AddWithValue("@num_plate", numPlate);
```

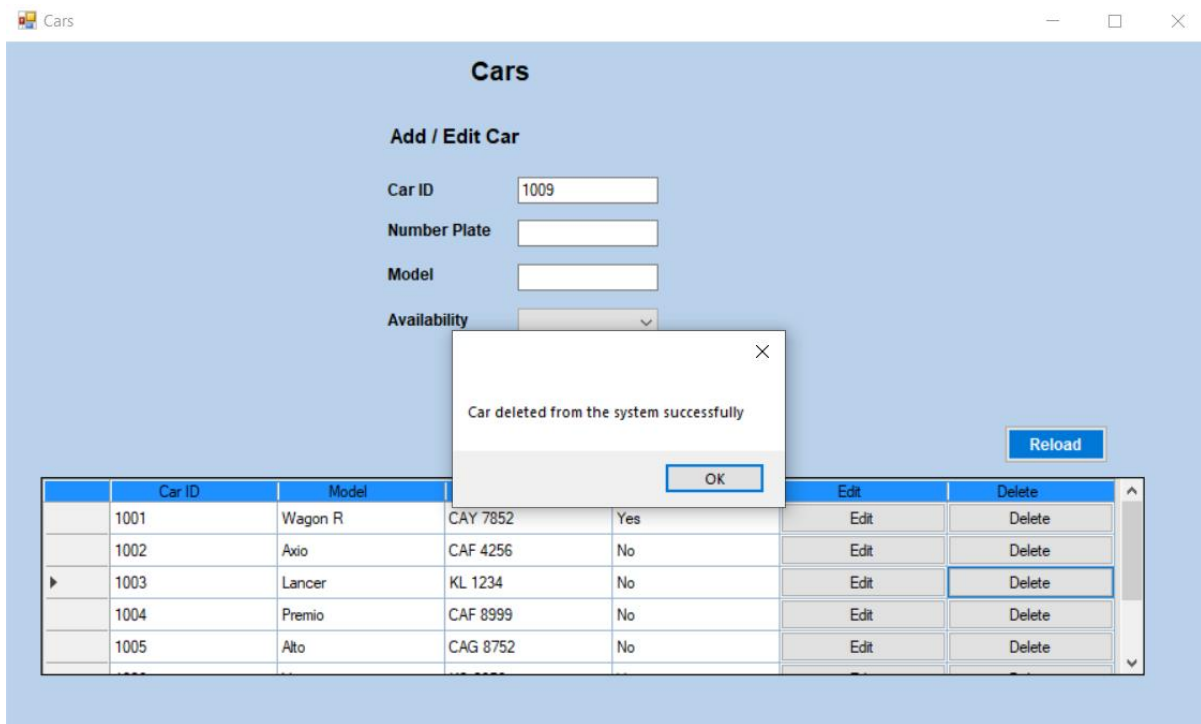
```

cmd.Parameters.AddWithValue("@model", model);
cmd.Parameters.AddWithValue("@available", available);
cmd.Parameters.AddWithValue("@reg_no", selectedNo);
cmd.ExecuteNonQuery();
con.Close();
MessageBox.Show("Car details updated successfully");
insert = true;
}
carIDTxt.Clear();
modelTxt.Clear();
npTxt.Clear();
avaTxt.SelectedIndex = -1;
generateNewCarID();
}

```

5.2.3 Delete a Car from the System

A car can be removed from the system by clicking on the delete button.



	Car ID	Model	Number Plate	Availability	Edit	Delete
▶	1001	Wagon R	CAY 7852	Yes	Edit	Delete
	1002	Axio	CAF 4256	No	Edit	Delete
	1004	Premio	CAF 8999	No	Edit	Delete
	1005	Alto	CAG 8752	No	Edit	Delete
	1006	Vitz	KO 2356	Yes	Edit	Delete

```

private void dataGridView1_CellContentClick(object sender,
DataGridViewCellEventArgs e)
{
    if (e.ColumnIndex == dataGridView1.Columns["editCol"].Index && e.RowIndex
    >= 0)
    {
        insert = false;
        selectedNo = dataGridView1.CurrentRow.Cells[0].Value.ToString();
        getCarDetails(selectedNo);
    }

    else if (e.ColumnIndex == dataGridView1.Columns["deleteCol"].Index &&
    e.RowIndex >= 0)
    {
        string deleteID = dataGridView1.CurrentRow.Cells[0].Value.ToString();
        con.Open();
        cmd = new SqlCommand("DELETE FROM cars WHERE reg_no = @reg_no", con);
        cmd.Parameters.AddWithValue("@reg_no", deleteID);
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Car deleted from the system successfully");
    }
}

```

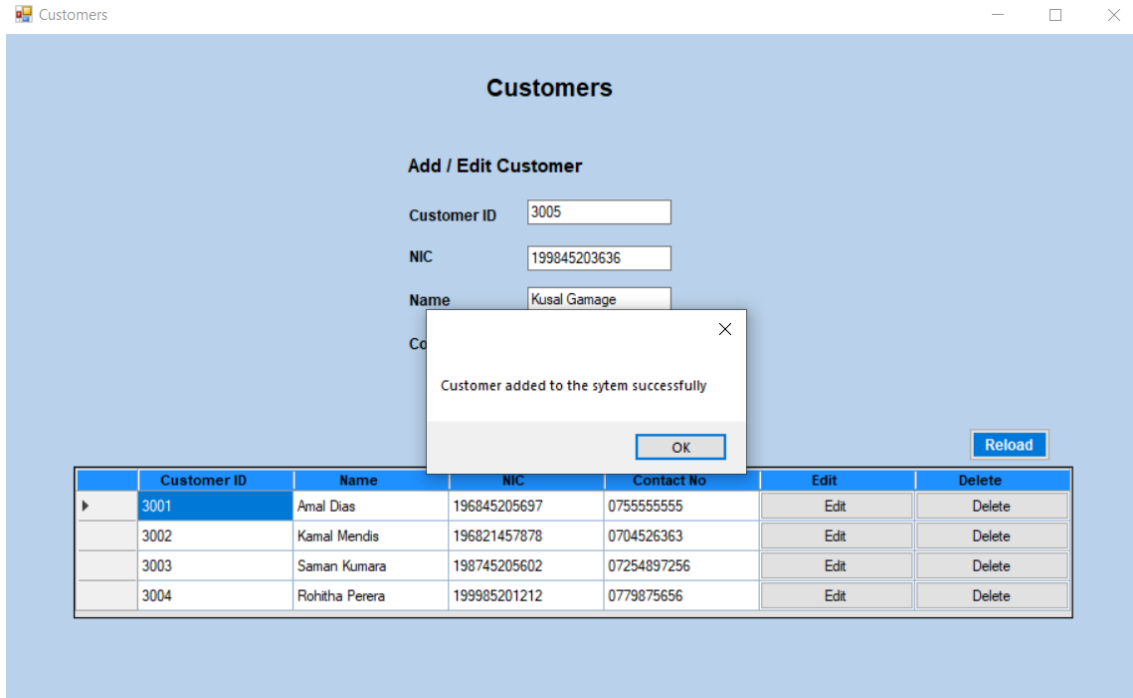
5.3 Customer Management

5.3.1 Add a New Customer to the System (Customer Registration)

A new customer can be added to the system by inputting the customer's name, NIC, contact number and clicking on the Save button. The Customer ID is automatically generated.

The screenshot shows a Windows application titled "Customers". It features a form for adding or editing a customer with fields for Customer ID (3005), NIC (199845203636), Name (Kusal Gamage), and Contact No (0714545488). A "Save" button is below the form. To the right of the form is a "Reload" button. Below the form is a table with the following data:

	Customer ID	Name	NIC	Contact No	Edit	Delete
▶	3001	Amal Dias	196845205697	0755555555	Edit	Delete
	3002	Kamal Mendis	196821457878	0704526363	Edit	Delete
	3003	Saman Kumara	198745205602	07254897256	Edit	Delete
	3004	Rohitha Perera	199985201212	0779875656	Edit	Delete



After a successful operation, a popup window will be shown to notify the user, and the new customer will now be visible on the table.

Customer ID	Name	NIC	Contact No	Edit	Delete
3003	Saman Kumara	198745205602	07254897256	Edit	Delete
3004	Rohitha Perera	199985201212	0779875656	Edit	Delete
3005	Kusal Gamage	199845203636	0714545488	Edit	Delete

5.3.2 Edit Customer Details

Details of an existing customer in the system can be updated by clicking on the edit button. The details of the selected car will be displayed, and after updating them, the user has to click the save button.

Customers

Customers

Add / Edit Customer

Customer ID3004

NIC199985201212

NameRohitha Perera

Contact No0770000011

Save

Reload

	Customer ID	Name	NIC	Contact No	Edit	Delete
	3003	Saman Kumara	198745205602	07254897256	Edit	Delete
▶	3004	Rohitha Perera	199985201212	0779875656	Edit	Delete
	3005	Kusal Gamage	199845203636	0714545488	Edit	Delete

After a successful operation, a popup window will be shown to notify the user, and the updated details will be visible on the table.

Customers

Customers

Add / Edit Customer

Customer ID3004

NIC199985201212

NameRohitha Perera

Con

Customer details updated successfully

OK

Reload

	Customer ID	Name	NIC	Contact No	Edit	Delete
	3003	Saman Kumara	198745205602	07254897256	Edit	Delete
▶	3004	Rohitha Perera	199985201212	0779875656	Edit	Delete
	3005	Kusal Gamage	199845203636	0714545488	Edit	Delete

	Customer ID	Name	NIC	Contact No	Edit	Delete
	3003	Saman Kumara	198745205602	07254897256	Edit	Delete
▶	3004	Rohitha Perera	199985201212	0770000011	Edit	Delete
	3005	Kusal Gamage	199845203636	0714545488	Edit	Delete

The following code is used for adding new customers to the system and editing the details of existing customers in the system.

```
private void custAddBtn_Click(object sender, EventArgs e)
{
    string regNo = custIDTxt.Text;
    string nic = nicTxt.Text;
    string name = nameTxt.Text;
    string contact = contactTxt.Text;

    con.Open();
    cmd = new SqlCommand("SELECT COUNT(*) FROM customers WHERE reg_no =
@reg_no", con);
    cmd.Parameters.AddWithValue("@reg_no", regNo);
    int count = (int)cmd.ExecuteScalar();
    con.Close();

    if (insert == true)
    {
        if (count > 0)
        {
            MessageBox.Show("Customer already exists with this registration
number.");
            return;
        }
        con.Open();
        cmd = new SqlCommand("INSERT INTO customers (reg_no, nic, name, contact)
VALUES (@reg_no, @nic, @name, @contact);", con);
        cmd.Parameters.AddWithValue("@reg_no", regNo);
        cmd.Parameters.AddWithValue("@nic", nic);
        cmd.Parameters.AddWithValue("@name", name);
        cmd.Parameters.AddWithValue("@contact", contact);
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Customer added to the sytem successfully");
    }
    else
    {
        con.Open();
        cmd = new SqlCommand("UPDATE customers SET nic = @nic, name = @name,
contact = @contact WHERE reg_no = @reg_no;", con);
        cmd.Parameters.AddWithValue("@nic", nic);
        cmd.Parameters.AddWithValue("@name", name);
        cmd.Parameters.AddWithValue("@contact", contact);
        cmd.Parameters.AddWithValue("@reg_no", selectedNo);
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Customer details updated successfully");
        insert = true;
    }
}
```

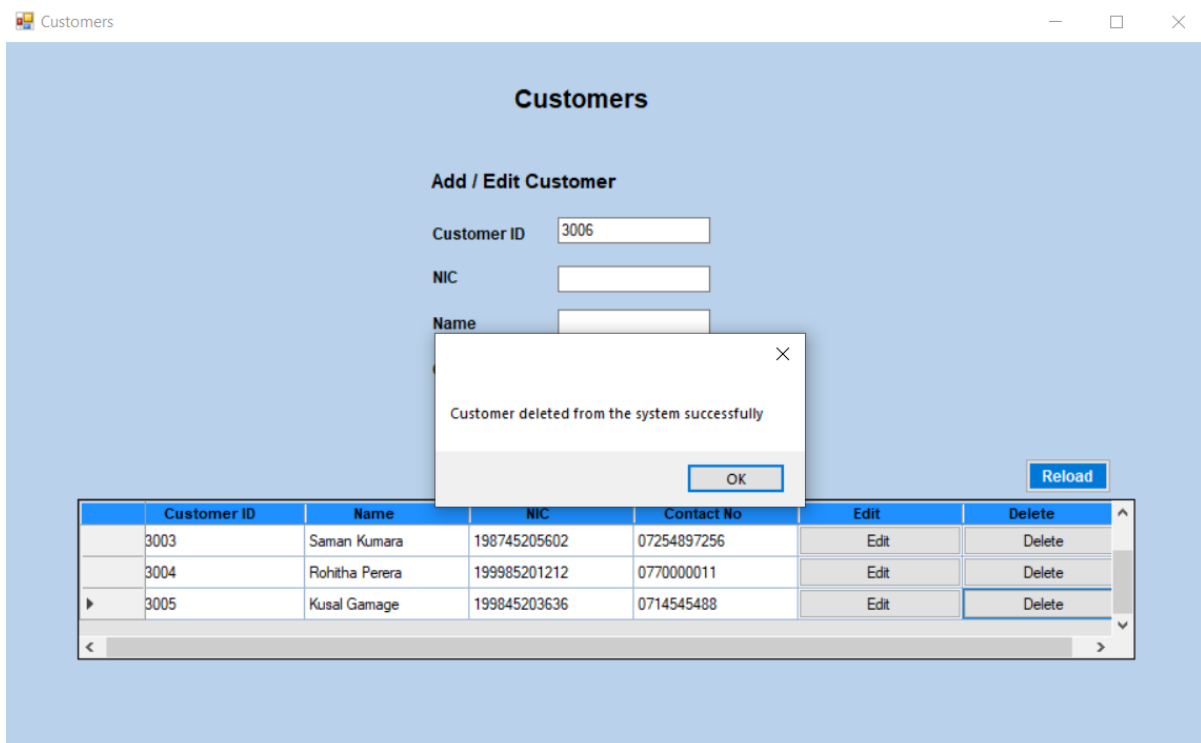
```

    }
    custIDTxt.Clear();
    nicTxt.Clear();
    nameTxt.Clear();
    contactTxt.Clear();
    generateNewCustomerID();
}

```

5.3.3 Delete a Customer from the System

A customer can be removed from the system by clicking on the delete button.



	Customer ID	Name	NIC	Contact No	Edit	Delete
▶	3001	Amal Dias	196845205697	0755555555	Edit	Delete
	3002	Kamal Mendis	196821457878	0704526363	Edit	Delete
	3003	Saman Kumara	198745205602	07254897256	Edit	Delete
	3004	Rohitha Perera	199985201212	0770000011	Edit	Delete

```

private void dataGridView1_CellContentClick(object sender,
DataGridViewCellEventArgs e)
{

```

```

    if (e.ColumnIndex == dataGridView1.Columns["editCol"].Index && e.RowIndex >=
0)
    {
        insert = false;
        selectedNo = dataGridView1.CurrentRow.Cells[0].Value.ToString();
        getCustomerDetails(selectedNo);
    }
    else if (e.ColumnIndex == dataGridView1.Columns["deleteCol"].Index &&
e.RowIndex >= 0)
    {
        string deleteID = dataGridView1.CurrentRow.Cells[0].Value.ToString();
        con.Open();
        cmd = new SqlCommand("DELETE FROM customers WHERE reg_no = @reg_no",
con);
        cmd.Parameters.AddWithValue("@reg_no", deleteID);
        cmd.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Customer deleted from the system successfully");
    }
}

```

5.4 Rental Management

5.4.1 Rental Commencement

A new rental can be added to the system by entering the relevant car and customer details, rental fee, and pickup and expected return dates. The rental ID is automatically generated.

Rentals

ADD NEW RENTAL

Rental ID

5010

Car ID

1007

Car Model

Honda Civic

Customer ID

3005

Customer Name

Charith Silva

Pickup Date

Monday, June 10, 2024

Return Date

Wednesday, June 12, 2024

Rental Fee (Rs)

4600

Confirm Rental

CONCLUDE RENTAL

Rental ID

Returned Date

Monday, June 10, 2024

Extra Charges (Rs)

Confirm Return

Reload

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed

ADD NEW RENTAL

Rental ID: 5010
Car ID: 1007
Car Model: Honda Civic
Customer ID: 3005
Customer Name: Charith Silva
Pickup Date: Monday, June 10, 2024
Return Date: Wednesday, June 12, 2024
Rental Fee (Rs): 4600

Confirm Rental

CONCLUDE RENTAL

Rental ID:
Returned Date: Monday, June 10, 2024
Extra Charges (Rs):

Confirm Return

Rental confirmed

OK

Reload

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed

After clicking on the confirm rental button, a confirmation popup will appear, and the rental record will be added to the table. The availability status of the rented car will change from "Yes" to "No" after confirmation.

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed
	5010	1007	3005	Charith Silva	10-06-2024	12-06-2024	4600	0	Return pending

The following code handles the process of adding a rental.

```
private void rentBtn_Click(object sender, EventArgs e)
{
    string rentID = rentIDTxt.Text;
    string carID = carCombo.SelectedItem.ToString();
    string customerID = customerCombo.SelectedItem.ToString();
    string customerName = custoNameTxt.Text;
    string pickupDate = pickDate.Value.Date.ToString("dd-MM-yyyy");
    string returnDate = retDate.Value.Date.ToString("dd-MM-yyyy");
    string fee = feeTxt.Text;
    int extraFee = 0;
    string rentStatus = "Return pending";

    con.Open();
```

```

        cmd = new SqlCommand("SELECT COUNT(*) FROM rentals WHERE rent_id =
@rent_id", con);
        cmd.Parameters.AddWithValue("@rent_id", rentID);
        int count = (int)cmd.ExecuteScalar();
        con.Close();

        if (count > 0)
        {
            MessageBox.Show("Rental already exists with this ID.");
            return;
        }

        con.Open();
        cmd = new SqlCommand("INSERT INTO rentals (rent_id, car_id, cust_id,
cust_name, pickup_date, return_date, fee, extra_fee, rent_status) VALUES
(@rent_id, @car_id, @cust_id, @cust_name, @pickup_date, @return_date, @fee,
@extra_fee, @rent_status)", con);
        cmd.Parameters.AddWithValue("@rent_id", rentID);
        cmd.Parameters.AddWithValue("@car_id", carID);
        cmd.Parameters.AddWithValue("@cust_id", customerID);
        cmd.Parameters.AddWithValue("@cust_name", customerName);
        cmd.Parameters.AddWithValue("@pickup_date", pickupDate);
        cmd.Parameters.AddWithValue("@return_date", returnDate);
        cmd.Parameters.AddWithValue("@fee", fee);
        cmd.Parameters.AddWithValue("@extra_fee", extraFee);
        cmd.Parameters.AddWithValue("@rent_status", rentStatus);
        cmd.ExecuteNonQuery();

        string sql1N = "UPDATE cars SET available = 'No' WHERE reg_no = @regno";
        SqlCommand cmdN = new SqlCommand(sql1N, con);
        cmdN.Parameters.AddWithValue("@regno", carID);
        cmdN.ExecuteNonQuery();
        con.Close();
        MessageBox.Show("Rental confirmed");

        rentIDTxt.Clear();
        feeTxt.Clear();
        custoNameTxt.Clear();
        carModTxt.Clear();
        statusLabel.Text = "";
        rentalCombo.Items.Clear();
        getAllRentals();
        generateNewRentalID();
        carCombo.SelectedIndex = -1;
        customerCombo.SelectedIndex = -1;
    }

```

If the user chooses an unavailable car for the rental, a notification will appear, and the confirm button and all other input fields will be locked.

ADD NEW RENTAL

Rental ID

Car ID

1004

This car is not available at the moment

Car Model

Customer ID

Customer Name

Pickup Date

Monday, June 10, 2024

Return Date

Monday, June 10, 2024

Rental Fee (Rs)

Confirm Rental

5.4.1 Conclude a Rental

A rental can be concluded by selecting the relevant ID and entering the return date and any extra charges, if applicable. After clicking the confirmation button, a popup will appear to notify the user, and the rental status will be updated from "Return pending" to "Completed." The availability status of the returned car will change from "No" to "Yes" after confirmation.

Rentals
— □ ×

ADD NEW RENTAL

Rental ID

Car ID

Car Model

Customer ID

Customer Name

Pickup Date

Monday, June 10, 2024

Return Date

Wednesday, June 12, 2024

Rental Fee (Rs)

Confirm Rental

CONCLUDE RENTAL

Rental ID

5010

Returned Date

Thursday, June 13, 2024

Extra Charges (Rs)

Confirm Return

Reload

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed
▶	5010	1007	3005	Charith Silva	10-06-2024	12-06-2024	4600	0	Return pending

ADD NEW RENTAL

Rental ID:

Car ID:

Car Model:

Customer ID:

Customer Name:

Pickup Date:

Return Date:

Rental Fee (Rs):

CONCLUDE RENTAL

Rental ID:

Returned Date:

Extra Charges (Rs):

Rental concluded successfully

Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed
5010	1007	3005	Charith Silva	10-06-2024	12-06-2024	4600	0	Return pending

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	560	Completed
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1001	3001	Amal Dias	24-05-2024	24-05-2024	3500	0	Completed
	5010	1007	3005	Charith Silva	10-06-2024	13-06-2024	4600	300	Completed

The following code handles the rental concluding process.

```
private void returnBtn_Click(object sender, EventArgs e)
{
    string returnDate = returnedDate.Value.Date.ToString("dd-MM-yyyy");
    string extraFee = extraTxt.Text;
    string selectedRentID = rentalCombo.SelectedItem.ToString();
    string selectedCarID = null;
    string newRentStatus = "Completed";

    cmd = new SqlCommand("SELECT * FROM rentals WHERE rent_id = '" +
selectedRentID + "'", con);
    con.Open();
    dr = cmd.ExecuteReader();
    if (dr.Read())
    {
        selectedCarID = dr["car_id"].ToString();
    }
    else
    {

```

```

        MessageBox.Show("No rental found for this ID");
    }
    con.Close();

    con.Open();
    SqlCommand cmdR = new SqlCommand("UPDATE rentals SET return_date =
@return_date, extra_fee = @extra_fee, rent_status = @rent_status WHERE rent_id =
@rentID", con);
    cmdR.Parameters.AddWithValue("@return_date", returnDate);
    cmdR.Parameters.AddWithValue("@extra_fee", extraFee);
    cmdR.Parameters.AddWithValue("@rent_status", newRentStatus);
    cmdR.Parameters.AddWithValue("@rentID", selectedRentID);
    cmdR.ExecuteNonQuery();

    SqlCommand cmdN = new SqlCommand("UPDATE cars SET available = 'Yes' WHERE
reg_no = @regno", con);
    cmdN.Parameters.AddWithValue("@regno", selectedCarID);
    cmdN.ExecuteNonQuery();
    con.Close();
    MessageBox.Show("Rental concluded successfully");

    extraTxt.Clear();
    rentalCombo.SelectedIndex = -1;
}

```

5.5 Payments Management

Total earnings for a given period can be calculated by entering the start and end IDs for that period. Additionally, an overview of all rental earnings can be viewed from the table.

Payments

Rental Earnings Overview

Start ID

End ID

Total earnings for this period :

	Rental ID	Rental Fee (Rs)	Extra Charges (Rs)	Total (Rs)
▶	5001	2500	400	2900
	5002	8900	2000	10900
	5003	7600	45	7645
	5004	10000	0	10000
	5005	2000	150	2150
	5006	3000	560	3560
	5008	2400	900	3300
	5009	3500	0	3500
	5010	4600	300	4900

Payments

Rental Earnings Overview

Start ID: 5002

End ID: 5009

Calculate

Total earnings for this period: Rs 41055

	Rental ID	Rental Fee (Rs)	Extra Charges (Rs)	Total (Rs)
▶	5001	2500	400	2900
	5002	8900	2000	10900
	5003	7600	45	7645
	5004	10000	0	10000
	5005	2000	150	2150
	5006	3000	560	3560
	5008	2400	900	3300
	5009	3500	0	3500
	5010	4600	300	4900

The code for calculating total earnings for a given period is as follows.

```
private decimal CalculateSum(int startId, int endId)
{
    decimal sumTotal = 0;
    cmd = new SqlCommand("SELECT fee, extra_fee FROM rentals WHERE rent_id
    BETWEEN @startId AND @endId", con);
    cmd.Parameters.AddWithValue("@startId", startId);
    cmd.Parameters.AddWithValue("@endId", endId);

    con.Open();
    dr = cmd.ExecuteReader();
    while (dr.Read())
    {
        decimal totalCharge = Convert.ToDecimal(dr[0]) +
        Convert.ToDecimal(dr[1]);
        sumTotal += totalCharge;
    }
    con.Close();

    return sumTotal;
}

private void calculateBtn_Click(object sender, EventArgs e)
{
    if (startCombo.SelectedItem != null && endCombo.SelectedItem != null)
    {
        int startId = Convert.ToInt32(startCombo.SelectedItem);
        int endId = Convert.ToInt32(endCombo.SelectedItem);
        if (startId < endId)
        {
            decimal sumTotal = CalculateSum(startId, endId);
        }
    }
}
```

```

        sumLabel.Text = "Total earnings for this period: Rs " +
sumTotal.ToString();
    }
    else
    {
        MessageBox.Show("Please select valid start and end IDs.");
    }
}
else
{
    MessageBox.Show("Please select valid start and end IDs.");
}
}
}

```

6. Appendix

6.1 Generating IDs

The following code generates new IDs for rentals. New IDs are created automatically in ascending order by checking the existing IDs in the database.

ADD NEW RENTAL

Rental ID

Car ID

Car Model

Customer ID

Customer Name

Pickup Date

Return Date

Rental Fee (Rs)

```

private void generateNewRentalID()
{
    con.Open();
    cmd = new SqlCommand("SELECT ISNULL(MAX(CAST(rent_id AS INT)), 5000) + 1
FROM rentals", con);
    int newID = (int)cmd.ExecuteScalar();
    con.Close();
    rentIDTxt.Text = newID.ToString();
}

```

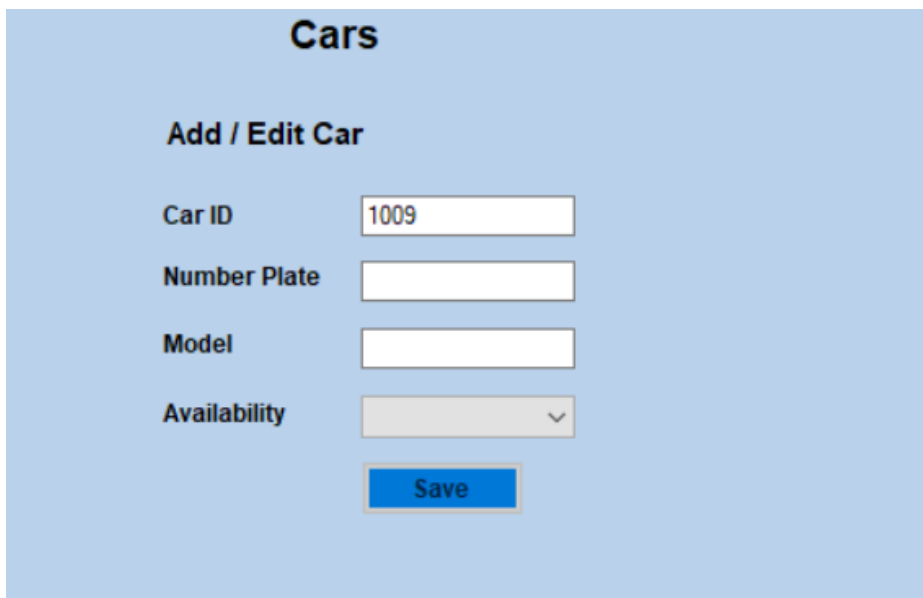
New IDs for customers and cars are also generated using the above approach.

The following code generates a new ID for a car when the car registration process starts.

```

private void generateNewCarID()
{
    con.Open();
    cmd = new SqlCommand("SELECT ISNULL(MAX(CAST(reg_no AS INT)), 1000) + 1 FROM
cars", con);
    int newID = (int)cmd.ExecuteScalar();
    con.Close();
    carIDTxt.Text = newID.ToString();
}

```



Cars

Add / Edit Car

Car ID

Number Plate

Model

Availability

The following code generates a new ID for a customer when the customer registration process starts.

```
private void generateNewCustomerID()
{
    con.Open();
    cmd = new SqlCommand("SELECT ISNULL(MAX(CAST(reg_no AS INT)), 3000) + 1 FROM
customers", con);
    int newID = (int)cmd.ExecuteScalar();
    con.Close();
    custIDTxt.Text = newID.ToString();
}
```



Customers

Add / Edit Customer

Customer ID

NIC

Name

Contact No

6.2 Load Data onto the Tables

6.2.1 Load Rental Data

The following code loads all rental data onto the rental table.

```
public void loadRentals()
{
    cmd = new SqlCommand("SELECT * FROM rentals", con);
    con.Open();
    dr = cmd.ExecuteReader();
    dataGridRental.Rows.Clear();
    while (dr.Read())
    {
        dataGridRental.Rows.Add(dr[0], dr[1], dr[2], dr[3], dr[4], dr[5], dr[6],
dr[7], dr[8]);
    }
    dr.Close();
    con.Close();
}
```

	Rent ID	Car ID	Customer ID	Customer Name	Pickup Date	Return Date	Rental Fee (Rs)	Extra Charges (Rs)	Rental Status
▶	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	0	Return pending
	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1007	3005	Charith Silva	12-06-2024	15-06-2024	3600	0	Return pending
	5010	1008	3006	Saranga Perera	14-06-2024	19-06-2024	7500	200	Completed

The same approach was used for loading data of customers and cars onto the tables.

6.2.2 Load Customer Data

This code is responsible for load all customer data onto the customer table.

```
public void loadCustomers()
{
    cmd = new SqlCommand("SELECT * FROM customers", con);
    con.Open();
    dr = cmd.ExecuteReader(); dataGridView1.Rows.Clear();
    while (dr.Read())
    {
        dataGridView1.Rows.Add(dr[0], dr[1], dr[2], dr[3]);
    }
    con.Close();
}
```

	Customer ID	Name	NIC	Contact No	Edit	Delete
▶	3001	Amal Dias	196845205697	0755555555	Edit	Delete
	3002	Kamal Mendis	196821457878	0704526363	Edit	Delete
	3003	Saman Kumara	198745205602	0779875448	Edit	Delete
	3004	Raksha Perera	199995201212	0772708488	Edit	Delete

6.2.3 Load Car Data

This code is responsible for load all the details of the cars onto the car table.

```
public void loadCars()
{
    cmd = new SqlCommand("SELECT * FROM cars", con);
    con.Open();
    dr = cmd.ExecuteReader(); dataGridView1.Rows.Clear();
    while (dr.Read())
    {
        dataGridView1.Rows.Add(dr[0], dr[1], dr[2], dr[3]);
    }
    con.Close();
}
```

	Car ID	Model	Number Plate	Availability	Edit	Delete
▶	1001	Suzuki Wagon R	CAY 7852	Yes	Edit	Delete
	1004	Toyota Premio	CAF 8999	No	Edit	Delete
	1005	Suzuki Alto	CAG 8752	No	Edit	Delete
	1007	Honda Civic	CAH 9999	No	Edit	Delete
	1008	Toyota Vitz	CAD 4556	Yes	Edit	Delete

6.2.4 Load Payments Data

This code is responsible for load all the details of the payments onto the payments table.

```
public void loadRentalCharges()
{
    cmd = new SqlCommand("SELECT rent_id, fee, extra_fee FROM rentals", con);
    con.Open();
    dr = cmd.ExecuteReader(); dataGridView1.Rows.Clear();
    while (dr.Read())
    {
        decimal totalCharge = Convert.ToDecimal(dr[1]) +
        Convert.ToDecimal(dr[2]);
        dataGridView1.Rows.Add(dr[0], dr[1], dr[2], totalCharge);
    }
    con.Close();
}
```

	Rental ID	Rental Fee (Rs)	Extra Charges (Rs)	Total (Rs)
▶	5001	2500	400	2900
	5002	8900	2000	10900
	5003	7600	45	7645
	5004	10000	0	10000
	5005	2000	150	2150
	5006	3000	0	3000
	5008	2400	900	3300
	5009	3600	0	3600
	5010	7500	200	7700

6.3 Initial Page Loading Process

6.3.1 Car Page Initialization

The car page is initialized by calling the following functions mentioned in earlier sections and establishing a connection to the database.

```
public CarRegister()
{
    InitializeComponent();
    loadCars();
    textChangedEvents();
    carAddBtn.Enabled = false;
    generateNewCarID();
}

SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=D:\\Projects\\CarRental\\CarRent
al\\CarRentalDB.mdf; Integrated Security=True;");
```

6.3.2 Customer Page Initialization

The customer page is initialized by calling the following functions mentioned in earlier sections and establishing a connection to the database.

```
public Customers()
{
    InitializeComponent();
    loadCustomers();
    textChangedEvents();
    custAddBtn.Enabled = false;
    generateNewCustomerID();
}

SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=D:\\Projects\\CarRental\\CarRent
al\\CarRentalDB.mdf; Integrated Security=True;");
```

6.3.3 Rental Page Initialization

The rental page is initialized by calling the following functions mentioned in earlier sections and establishing a connection to the database.

```
public RentCar()
{
    InitializeComponent();
    getAllCars();
    getAllCustomers();
    getAllRentals();
    loadRentals();
    rentBtn.Enabled = false;
    generateNewRentalID();
}

SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=D:\\Projects\\CarRental\\CarRent
al\\CarRentalDB.mdf; Integrated Security=True;");
```

6.3.4 Payments Page Initialization

The rental page is initialized by calling the following functions mentioned in earlier sections and establishing a connection to the database.

```
public Payments()
{
    InitializeComponent();
    loadRentalCharges();
    loadRentalIDs();
}

SqlConnection con = new SqlConnection("Data
Source=(LocalDB)\\MSSQLLocalDB;AttachDbFilename=D:\\Projects\\CarRental\\CarRent
al\\CarRentalDB.mdf; Integrated Security=True;");
```

6.4 Enable Save Buttons

6.4.1 Enable Customer Save Button

This is the code to enable the "Save" button when registering new customers. It will only be enabled after all required fields are filled.

```
private void textChangedEvents()
{
    custIDTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    nicTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    nameTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    contactTxt.TextChanged += new EventHandler(TextFields_TextChanged);
}

private void TextFields_TextChanged(object sender, EventArgs e)
{
    custAddBtn.Enabled = !string.IsNullOrEmpty(custIDTxt.Text) &&
        !string.IsNullOrEmpty(nicTxt.Text) &&
        !string.IsNullOrEmpty(nameTxt.Text) &&
        !string.IsNullOrEmpty(contactTxt.Text);
}
```

The image displays two versions of a web form titled "Add / Edit Customer". Both forms have a light blue background and contain four text input fields: "Customer ID", "NIC", "Name", and "Contact No", followed by a blue "Save" button.

- Left Form:** The "Customer ID" field contains the value "3007". The "NIC", "Name", and "Contact No" fields are empty. The "Save" button is disabled (grayed out).
- Right Form:** The "Customer ID" field contains "3007", "NIC" contains "199323205441", "Name" contains "Kusal Perera", and "Contact No" contains "0714542489". The "Save" button is enabled (blue).

6.4.2 Enable Car Save Button

This is the code to enable the "Save" button when registering new cars. It will only be enabled after all required fields are filled.

```
private void textChangedEvents()
{
    carIDTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    npTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    modelTxt.TextChanged += new EventHandler(TextFields_TextChanged);
    avaTxt.TextChanged += new EventHandler(TextFields_TextChanged);
}

private void TextFields_TextChanged(object sender, EventArgs e)
{
    carAddBtn.Enabled = !string.IsNullOrEmpty(carIDTxt.Text) &&
        !string.IsNullOrEmpty(npTxt.Text) &&
        !string.IsNullOrEmpty(modelTxt.Text) &&
        !string.IsNullOrEmpty(avaTxt.Text);
}
```

The image shows two versions of a web form titled "Add / Edit Car". The form contains four input fields: "Car ID", "Number Plate", "Model", and "Availability", followed by a "Save" button. In the left version, "Car ID" is 1009, "Number Plate" is empty, "Model" is empty, and "Availability" has a dropdown arrow. In the right version, "Car ID" is 1009, "Number Plate" is "CAF 1423", "Model" is "Toyota Axio", and "Availability" is "Yes".

6.5 Prevent Rental Conflicts

When a user tries to start a new rental for a car that is not available, a message will be displayed, and all the fields along with the "Confirm Rental" button will be locked. The following code is responsible for this task.

```
private void carCombo_SelectedIndexChanged(object sender, EventArgs e)
{
    if (carCombo.SelectedIndex != -1)
    {
        string selectedCarRegNo = carCombo.SelectedItem.ToString();
        cmd = new SqlCommand("SELECT * FROM cars WHERE reg_no = '" +
selectedCarRegNo + "'", con);
        con.Open();
        dr = cmd.ExecuteReader();
        if (dr.Read())
        {
            carModTxt.Text = dr["model"].ToString();
            string status;
            status = dr["available"].ToString();
            if (status == "No")
            {
                statusLabel.Text = "This car is not available at the moment";
                customerCombo.Enabled = false;
                custoNameTxt.Enabled = false;
                pickDate.Enabled = false;
                retDate.Enabled = false;
                feeTxt.Enabled = false;
                rentBtn.Enabled = false;
            }
            else
            {
                statusLabel.Text = "";
                customerCombo.Enabled = true;
                custoNameTxt.Enabled = true;
                pickDate.Enabled = true;
            }
        }
    }
}
```

```

        retDate.Enabled = true;
        feeTxt.Enabled = true;
        rentBtn.Enabled = true;
    }
}
else
{
    MessageBox.Show("No car found");
}
con.Close();
}
}

```

6.6 Database Management

Microsoft SQL Server is used for managing the database, with separate databases utilized for user login, cars, customers, and rentals.

- User login database

dbo.login [Data]			
Max Rows: 1000			
	Id	username	password
	1	E2248422	pass123
	NULL	NULL	NULL

- Car database

dbo.cars [Data]				
	reg_no	model	num_plate	available
▶	1001	Suzuki Wagon R	CAY 7852	Yes
	1004	Toyota Premio	CAF 8999	No
	1005	Suzuki Alto	CAG 8752	No
	1007	Honda Civic	CAH 9999	No
	1008	Toyota Vitz	CAD 4556	Yes
⊗	NULL	NULL	NULL	NULL

- Customer database

dbo.customers [Data]				
	reg_no	name	nic	contact
▶	3001	Amal Dias	196845205697	0755555555
	3002	Kamal Mendis	196821457878	0704526363
	3003	Saman Kumara	198745205602	0779875448
	3004	Rohitha Perera	199985201212	0723708498
	3005	Charith Silva	198820204525	0789896999
	3006	Saranga Perera	200145454525	0714142488
⊗	NULL	NULL	NULL	NULL

- Rental Database

dbo.rentals [Data]									
	rent_id	car_id	cust_id	cust_name	pickup_date	return_date	fee	extra_fee	rent_status
	5001	1003	3002	Kamal Mendis	10-05-2024	14-05-2024	2500	400	Completed
	5002	1001	3001	Amal Dias	24-05-2024	24-05-2024	8900	2000	Completed
	5003	1003	3002	Kamal Mendis	20-05-2024	23-05-2024	7600	45	Completed
	5004	1001	3003	Saman Kumara	26-05-2024	29-05-2024	10000	0	Completed
	5005	1004	3001	Amal Dias	24-05-2024	26-05-2024	2000	150	Completed
	5006	1005	3003	Saman Kumara	24-05-2024	28-05-2024	3000	0	Return pending
▶	5008	1004	3002	Kamal Mendis	24-05-2024	02-06-2024	2400	900	Completed
	5009	1007	3005	Charith Silva	12-06-2024	15-06-2024	3600	0	Return pending
	5010	1008	3006	Saranga Perera	14-06-2024	19-06-2024	7500	200	Completed
⊗	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

7. References

- [1] Amey Thakur. “Car Rental System”. Internet: https://www.researchgate.net/publication/353174644_Car_Rental_System, Jul. 2021 [May. 18, 2024]
- [2] Voyage Labs. “Car Rental System”. Internet: <https://www.voyagelabs.co/blog/car-rental-system>, Nov. 03, 2023 [May. 20, 2024]
- [3] Unknown Author. “Exploring the Essence of Car Rental Management Systems”. Internet: <https://www.businessfirstonline.co.uk/advice/exploring-the-essence-of-car-rental-management-systems/>, Jan. 11, 2024 [May. 20, 2024]